Document number:	P0013R0
Date:	2015-07-05
Project:	Programming Language C++, Library Evolution Working Group
Reply-to:	Jonathan Wakely <cxx@kayari.org>

Logical Operator Type Traits

I. Table of Contents

II. Introduction

I propose three new type traits for performing logical operations with other traits, `and_`, `or_`, and `not_`, corresponding to the operators `&&`, `||`, and `!` respectively. These utilities are used extensively in libstdc++ and are valuable additions to any metaprogramming toolkit.

```
// logical conjunction:
template<class... B> struct and_;

// logical disjunction:
template<class... B> struct or_;

// logical negation:
template<class B> struct not_;
```

III. Motivation and Scope

The proposed traits apply a logical operator to the result of one or more type traits, for example the specialization `and_<is_copy_constructible<T>, is_copy_assignable<T>>` has a BaseCharacteristic of `true_type` only if `T` is copy constructible and copy assignable, or in other words it has a BaseCharacteristic equivalent to `bool_constant<is_copy_constructible_v<T> && is_copy_assignable_v<T>>`.

The traits are especially useful when dealing with variadic templates where you want to apply the same predicate or transformation to every element in a parameter pack, for example `or_<is_nothrow_default_constructible<T>...>` can be used to determine whether at least one of the types in the pack `T` has a non-throwing default constructor.

Practicing metaprogrammers often consider making more of our unary type traits and/or type relationship traits variadic; a request that arises seemingly very often is having something like 'are_same' which would be a variadic `is_same`. **The proposed conjunction trait allows turning *any* trait into a variadic trait, without adding a plethora of variadic counterparts for individual traits that we already have.** The disjunction and negation complete the set.

Quoth Ville: "`__and_`, `__or_` and `__not_` as they are already available in libstdc++ are an absolute godsend, not just for a library writer, but for any user who needs to do metaprogramming with parameter packs. Boost.MPL has shipped similar facilities for a very long time, and it's high time we standardize these simple utilities."

To demonstrate how to use `and_` with `is_same` to make an 'all_same' trait, constraining a variadic function template so that all arguments have the same type can be done using `and_<is_same<T, Ts>...>`, for example:

```
// Accepts one or more arguments of the same type.
template<typename T, typename... Ts>
  enable_if_t< and_v<is_same<T, Ts>...> >
  func(T, Ts...)
  { }
```

For the sake of clarity this function doesn't do perfect forwarding, but if the parameters were forwarding references the

constraint would only be slightly more complicated: `and_v<is_same<decay_t<T>, decay_t<Ts>>...>`.

Constraining all elements of a parameter pack to a specific type can be done similarly:

```
// Accepts zero or more arguments of type int.
template<typename... Ts>
  enable_if_t< and_v<is_same<int, Ts>...> >
  func(Ts...)
  { }
```

Of course the three traits can be combined to form arbitrary predicates, the specialization `and_<or_<foo, bar>, not_<baz>>` corresponds to `(foo::value || bar::value) && !baz::value`.

IV. Impact On The Standard

This is a pure addition with no dependency on anything that isn't already in the 2014 standard. I propose it for inclusion in the Library Fundamentals TS rather than the International Standard. If the changes are applied to the C++17 working paper instead then `not_` could be changed to use `bool_constant`.

V. Design Decisions

In this proposal the class templates `and_` and `or_` derive from their arguments. An alternative design would be to force a *BaseCharacteristic* of `true_type` or `false_type`, i.e. instead of the proposed design:

```
  template<class B1> struct and_<B1> : B1 { };
```

we could specify it as:

```
  template<class B1> struct and_<B1> : bool_constant<B1::value> { };
```

I believe the former is more flexible and preserves more information.

We could require the template arguments themselves to have a *BaseCharacteristic* of `true_type` or `false_type` but I think that would be an unnecessary restriction, they only really require `B::value` to be convertible to bool. As proposed the traits will work with user-defined traits that have a nested `value` member of an enumeration type and other forms of trait that don't derive from a specialization of `integral_constant`.

The variable templates `and_v`, `or_v`, and `not_v` do not exist in libstdc++, largely because these traits predate G++'s support for variable templates by several years. However the examples above demonstrate their usefulness and adding them is consistent with the other variable templates proposed by N3854.

Many uses of these traits with parameter packs can be replaced by fold expressions, for example `and_v<T...>` can be replaced by `(true && ... && T::value)`, however the fold expression will instantiate `T::value` for every element of the pack, whereas the the proposed behaviour of `and_` and `or_` only instantiates as many elements of the pack as necessary to determine the answer, i.e. they perform short-circuiting with regard to instantiations. This short-circuiting makes `and_<is_copy_constructible<T>, is_copy_assignable<T>>` potentially cheaper to instantiate than the logically equivalent `bool_constant<is_copy_constructible_v<T> && is_copy_assignable_v<T>>`.

Efficiency aside, the short-circuiting property allows these traits to be used in contexts that would be more difficult otherwise. If we have a trait `is_foo` such that `is_foo<T>::value` is ill-formed unless `T` is a class type, the expression `is_class_v<T> && is_foo<T>::value` would be unsafe and must be replaced by something using a extra level of indirection to ensure `is_foo::value` is only instantiated when valid, such as `conditional_t<is_class_v<T>, is_foo<T>, false_type>::value`. This is almost precisely what `and_v<is_class<T>, is_foo<T>>` expands to, but `and_v` expresses the intention more clearly.

The traits are given the obvious names, adjusted to avoid clashing with the keywords of the same names. Another option would have been `static_and` but the context and angle brackets should be enough to make it obvious these are templates that are evaluated at compile-time, and the shorter names are easier to read.

VI. Technical Specification

> For the purposes of SG10, I recommend a feature-testing macro named `__cpp_lib_experimental_logical_traits`.

Add to the synopsis in [meta.type.synop]:

```
// [meta.logical], logic operator traits:
```

```
template<class... B> struct and_;
template<class... B> constexpr bool and_v = and_<B...>::value;
template<class... B> struct or_;
template<class... B> constexpr bool or_v = or_<B...>::value;
template<class B> struct not_;
template<class B> constexpr bool not_v = not_<B>::value;
```

Add a new subclause in [meta]:

3.3.x Logical operator traits [meta.logical]

This subclause describes type traits for applying logical operators to other type traits.

```
template<class... B> struct and_ : see below { };

template<class... B> constexpr bool and_v = and_<B...>::value;
```

The class template `and_` forms the logical conjunction of its template type arguments. Every template type argument shall be usable as a base class and shall have a member `value` which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

The BaseCharacteristic of a specialization `and_<B1, ..., BN>` is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value == false`, or if every `Bi::value != false` the BaseCharacteristic is `BN`. [*Note:* This means a specialization of `and_` does not necessarily have a BaseCharacteristic of either `true_type` or `false_type`. — *end note*]

For a specialization `and_<B1, ..., BN>` if there is a template type argument `Bi` with `Bi::value == false` then instantiating `and_<B1, ..., BN>::value` does not require the instantiation of `Bj::value` for `j > i`. [*Note:* This is analogous to the short-circuiting behavior of `&&`. — *end note*]

```
template<class... B> struct or_ : see below { };

template<class... B> constexpr bool or_v = or_<B...>::value;
```

The class template `or_` forms the logical disjunction of its template type arguments. Every template type argument shall be usable as a base class and shall have a member `value` which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

The BaseCharacteristic of a specialization `or_<B1, ..., BN>` is the first type `Bi` in the list `false_type, B1, ..., BN` for which `Bi::value != false`, or if every `Bi::value == false` the BaseCharacteristic is `BN`. [*Note:* This means a specialization of `or_` does not necessarily have a BaseCharacteristic of either `true_type` or `false_type`. — *end note*]

For a specialization `or_<B1, ..., BN>` if there is a template type argument `Bi` with `Bi::value != false` then instantiating `or_<B1, ..., BN>::value` does not require the instantiation of `Bj::value` for `j > i`. [*Note:* This is analogous to the short-circuiting behavior of `||`. — *end note*]

```
template<class B> struct not_ : integral_constant<bool, !B::value> { };

template<class B> constexpr bool not_v = not_<B>::value;
```

The class template `not_` forms the logical negation of its template type argument. The type `not_<B>` is a UnaryTypeTrait with a BaseCharacteristic of `integral_constant<bool, !B::value>`.

VII. Sample Implementation

Example implementations of `and_` and `or_` based on Daniel Krügler's code in libstdc++ are shown here:

```
template<class...> struct and_; // not defined

template<> struct and_<> : true_type { };

template<class B1> struct and_<B1> : B1 { };

template<class B1, class B2>
  struct and_<B1, B2>
  : conditional_t<B1::value, B2, B1>
  { };

template<class B1, class B2, class B3, class... Bn>
  struct and_<B1, B2, B3, Bn...>
  : conditional_t<B1::value, and_<B2, B3, Bn...>, B1>
  { };
```

```
template<class...> struct or_; // not defined

template<> struct or_<> : false_type { };

template<class B1> struct or_<B1> : B1 { };

template<class B1, class B2>
  struct or_<B1, B2>
  : conditional_t<B1::value, B1, B2>
  { };

template<class B1, class B2, class B3, class... Bn>
  struct or_<B1, B2, B3, Bn...>
  : conditional_t<B1::value, B1, or_<B2, B3, Bn...>>
  { };
```

VIII. Acknowledgements

Thanks to Daniel Krügler for the original implementation in libstdc++ which inspired this proposal, and to Ville Voutilainen for nudging me to write it up and for providing part of the motivation section.

IX. References

N3854, Variable Templates For Type Traits, by Stephan T. Lavavej.